HTTP Headers

1. Introduction:

- HTTP headers are **key-value pairs** that carry additional information in both HTTP requests and responses.
- They provide **metadata** about the request or response, such as content type, length, encoding, authentication, and caching policies.
- Headers enable communication between client and server beyond the actual data transfer.

2. Purpose of HTTP Headers:

- Convey essential **metadata** for interpretation and processing of HTTP messages.
- Control **browser behavior**, **security**, and **content delivery**.
- Enable features such as **compression, caching, authentication, and cross-origin requests**.
- Used in both:
    - **Request headers:** Sent by the client to the server.
    - **Response headers:** Sent by the server back to the client.

3. Structure of an HTTP Header:

Each header consists of:

- A **field name** (case-insensitive)
- A **colon (:)**
- A **field value**

Example:
```
Host: www.example.com
Content-Type: text/html
User-Agent: Mozilla/5.0
```

Multiple headers are separated by new lines and are placed between the start line and the body of the message.

4. Types of HTTP Headers:

HTTP headers are grouped into several categories:

1. **General Headers**
   - Applicable to both request and response messages.
   - Provide general information about the message itself.
   - Examples:
     Cache-Control: no-cache
     Connection: keep-alive
     Date: Wed, 15 Oct 2025 12:00:00 GMT

2. **Request Headers**
   - Provide additional information about the client or resource being requested.
   - Examples:
     Host: www.example.com
     User-Agent: Mozilla/5.0
     Accept: text/html
     Accept-Language: en-US
     Referer: https://www.google.com

3. **Response Headers**
   - Provide information about the server and the response being sent.
   - Examples:
     Server: Apache/2.4.1 (Unix)
     Set-Cookie: sessionId=abcd1234
     Content-Type: text/html
     Content-Length: 1024
     Location: https://www.example.com/newpage

4. **Entity Headers**
   - Contain information about the body of the message.
   - Examples:
     Content-Type: application/json
     Content-Length: 256
     Content-Encoding: gzip
     Last-Modified: Wed, 15 Oct 2025 09:00:00 GMT

5. Commonly Used Request Headers:

Header	Description	Example
Host	Specifies the domain name of the server	Host: www.example.com
User-Agent	Identifies the client software	User-Agent: Chrome/120.0
Accept	Specifies acceptable content types	Accept: text/html, application/json
Accept-Language	Preferred language of response	Accept-Language: en-US
Referer	URL of the page making the request	Referer: https://www.google.com
Authorization	Contains credentials for authentication	Authorization: Basic QWxhZGRpbjpPcGVuU2VzYW1l
Content-Type	Type of content sent in body	Content-Type: application/x-www-form-urlencoded
Cookie	Sends stored cookies to the server	Cookie: sessionId=abcd1234

6. Commonly Used Response Headers

Header	Description	Example
Server	Identifies the server software	Server: Apache/2.4.1
Set-Cookie	Sends cookies to the client	Set-Cookie: userId=789; Path=/
Content-Type	Type of response data	Content-Type: text/html; charset=UTF-8
Content-Length	Size of response body	Content-Length: 1024
Location	Redirects client to a new URL	Location: https://www.example.com/home
Access-Control-Allow-Origin	Defines allowed origins (CORS)	Access-Control-Allow-Origin: *
Cache-Control	Defines caching behavior	Cache-Control: no-store, max-age=0
Expires	Sets an expiry date for cached content	Expires: Wed, 15 Oct 2025 14:00:00 GMT

7. Example: HTTP Request with Headers

```
GET /index.html HTTP/1.1
 Host: www.example.com
 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
 Accept: text/html
 Accept-Language: en-US
 Connection: keep-alive
```

Explanation:

- The client requests the file “index.html” from the server.
- It sends metadata such as browser type and preferred language.
- The connection is kept alive for further requests.

8. Example: HTTP Response with Headers

```
HTTP/1.1 200 OK
 Date: Wed, 15 Oct 2025 12:05:00 GMT
 Server: Apache/2.4.1 (Unix)
 Content-Type: text/html
 Content-Length: 1248
 Connection: keep-alive
 Set-Cookie: theme=light; Path=/; Expires=Thu, 16 Oct 2025 12:05:00 GMT

<html> <body> <h1>Welcome to Example.com</h1> </body> </html>
```

Explanation:

- The status line indicates a successful response (200 OK).
- The headers describe the content type, size, and session cookies.
- The body contains the HTML content.

9. Security and Control Headers

1. **Security Headers**
    - **Strict-Transport-Security:** Forces HTTPS connections only.
      Example: Strict-Transport-Security: max-age=31536000; includeSubDomains
    - **X-Content-Type-Options:** Prevents MIME-sniffing attacks.
      Example: X-Content-Type-Options: nosniff
    - **Content-Security-Policy:** Restricts sources of scripts and resources.
      Example: Content-Security-Policy: default-src 'self'

2. **Control Headers:**
    - **Cache-Control:** Specifies caching behavior.
      Example: Cache-Control: no-cache, no-store, must-revalidate
    - **Pragma:** Used for backward compatibility with HTTP/1.0.
      Example: Pragma: no-cache
    - **ETag:** Used for efficient caching by identifying content versions.
      Example: ETag: "abcd1234"

10. Custom and Experimental Headers

- Developers can define **custom headers** for specific applications.
- These begin with **"X-"** by convention.
- Example:
  X-Request-ID: 987654321
  X-Frame-Options: DENY
- Though modern standards discourage "X-" prefixes, they remain in legacy use.

11. Importance of HTTP Headers

- Define how browsers and servers handle data.
- Enable **security mechanisms**, **caching**, **compression**, and **session control**.
- Aid in **debugging** and **troubleshooting** using browser developer tools.
- Essential for **API development**, **content negotiation**, and **cross-domain communication**.

Summary

- HTTP headers are essential for **metadata exchange** between client and server.
- They define how the message is transmitted, cached, and secured.
- Understanding headers is vital for **web development**, **API design**, and **network security**.
- Correct usage improves **performance**, **security**, and **user experience**.